DUOC UC · SAN JOAQUÍN

SKIP DUOC UC — GUÍA MAESTRA DE ESTUDIO

Defensa de Proyecto de Título · Escuela de Informática y Telecomunicaciones

Asignatura: Taller de Proyecto (APT122)
Proyecto: SKIP Duoc UC (Click & Collect)
Versión: Cierre Fase 3 (Documento Oficial)

1. ASIGNACIÓN DE ROLES Y DOMINIO TEMÁTICO (DIAPOSITIVA 2)

Cada integrante del equipo debe responder con total autoridad técnica y propiedad sobre su módulo:

Integrante	Rol Oficial	Módulos y Temas Obligatorios en la Defensa
Martín Aburto	Fullstack Lead & PM	Arquitectura general desacoplada, modelo relacional en PostgreSQL, políticas RLS, integración de pasarelas de pago (Webpay Sandbox y BancoEstado CuentaRUT) y coordinación metodológica.
Andrés Barría	Backend & Mobile Developer	Aplicación móvil en React Native + Expo, gestión reactiva de estado con Zustand, regla de carrito mono-tienda y escuchador global de eventos WebSocket en tiempo real.
Elías Orellana	Frontend Web Developer	Consola web administrativa en React 19 + Vite, mantenedor de catálogo de productos, modificadores y agregados, control de stock y reportes de recaudación.
Juan Pizarro	Frontend Web & QA	Tablero Kanban de cocina, código alfanumérico SJ-XXX, ticket térmico de 80 mm, los 4 ciclos formales de pruebas y matriz de trazabilidad de requisitos (RTM).

2. DICCIONARIO DE TECNICISMOS DEL PROYECTO (GLOSARIO EXIGIDO)

Arquitectura y Backend

BaaS (Backend as a Service)

Modelo serverless donde un proveedor administrado (Supabase) provee base de datos PostgreSQL, autenticación, storage y tiempo real. Elimina la necesidad de programar y mantener un servidor API monolítico en Express/NestJS.

Supabase Realtime (CDC - Change Data Capture)

Tecnología basada en **WebSockets** que escucha el registro de replicación transaccional de PostgreSQL (WAL). Al pagar un pedido, emite un evento `INSERT/UPDATE` y el Kanban de cocina lo renderiza en menos de 0.5 segundos sin recargar.

Polling vs. WebSockets

El *polling* tradicional pregunta al servidor cada 3 segundos si hay pedidos nuevos, saturando la red. **WebSockets** mantiene un canal TCP bidireccional permanente y solo transmite datos cuando ocurre un evento real.

GoTrue & Tokens JWT (JSON Web Tokens)

Mecanismo de autenticación sin estado (*stateless*). Al iniciar sesión, Supabase emite un token firmado criptográficamente que contiene el ID de usuario (`uid`), rol (`user_role`) y expiración, transmitiéndose en la cabecera `Authorization: Bearer`.

Base de Datos y Seguridad

Row Level Security (RLS)

Característica nativa de PostgreSQL que evalúa permisos fila por fila mediante sentencias SQL (`USING` y `WITH CHECK`). Garantiza el aislamiento multitenant: un locatario solo puede consultar pedidos donde `locatario_id = auth.uid()`.

Error de Recursión en RLS (PostgreSQL 42P17)

Ocurre cuando una política de seguridad consulta la misma tabla en un subquery para verificar permisos, generando un bucle infinito que bloquea la base de datos. Se resolvió creando funciones `SECURITY DEFINER` y desanidando las reglas.

UUID v4 (Universally Unique Identifier)

Identificador aleatorio de 128 bits. Se utilizó en lugar de IDs auto-incrementales (1, 2, 3...) para impedir ataques de enumeración (evitar que un usuario deduzca el volumen de pedidos de un local o acceda a órdenes ajenas cambiando el ID numérico).

Frontend, Pasarelas y Hardware

Zustand con Persist Middleware

Gestor de estado global liviano y atómico para React. Mantiene el carrito en memoria y sincronizado en `SecureStore` (móvil) o `localStorage` (web), evitando que el alumno pierda su comanda si la app se cierra.

Regla de Negocio "Carrito Mono-Tienda"

Restricción deliberada del sistema: un pedido no puede mezclar productos de locales distintos. Cada local cuenta con cocina, capacidad de mesón y tiempos de preparación (SLA) independientes.

Transbank Webpay Plus Sandbox (Comercio 597055555532)

Ambiente oficial de pruebas de Transbank que simula transacciones reales de débito Redcompra y crédito bajo protocolo HTTPS REST tokenizado.

BancoEstado CuentaRUT (Open Banking Dev)

Pasarela directa simulada mediante validación de Clave Pass. Ahorra al locatario el 2.5% de comisión por cobro en terminal POS físico tradicional.

Ticket Térmico POS 80 mm

Modal de impresión diseñado bajo estándar de comandas térmicas con código de barras `CODE128` y tipografía monoespaciada para locales sin pantalla en la línea de cocina.

3. CUESTIONARIO OFICIAL DE DEFENSA DUOC UC (RESPUESTAS TÉCNICAS)

Respuestas exactas para las 5 áreas y preguntas complementarias que evalúa la comisión:

Bloque 1: Arquitectura de Software y Diseño del Sistema

1.1 ¿Qué tipo de arquitectura de software utilizaron en su proyecto y por qué la seleccionaron?

"Utilizamos una **arquitectura desacoplada basada en Backend as a Service (BaaS) Serverless** sobre Supabase, articulando un modelo Cliente-Servidor multicliente (App Móvil en React Native + Expo y Consola Web en React 19 + Vite). La seleccionamos porque un sistema de compras universitarias exige sincronización en tiempo real y alta disponibilidad en horas punta; el modelo BaaS nos entregó un motor PostgreSQL 15 industrial con WebSockets y autenticación nativa, reduciendo costos de mantenimiento de infraestructura y permitiendo enfocar los recursos en la experiencia de usuario y lógica de negocio."

1.2 ¿Qué ventajas y desventajas presenta la arquitectura seleccionada frente a otras alternativas?

Ventajas: Desacoplamiento total (si la consola web se apaga, la app móvil sigue operando y los pedidos se encolan en BD); latencia sub-segundo (< 500 ms) en cocina vía WebSockets CDC; y seguridad declarativa a nivel de datos (RLS).
Desventajas: Acoplamiento al ecosistema de Supabase (*vendor lock-in*) y menor personalización a bajo nivel de sockets comparado con un servidor en C++ o Go puro.

1.3 ¿Cómo se comunican los componentes del sistema (frontend, backend y base de datos)?

• **App Móvil → Supabase:** Peticiones HTTPS bajo protocolo REST (PostgREST API) enviando el token JWT en la cabecera `Authorization` para autenticación, consulta de catálogo y creación de pedidos.
• **Supabase → Terminal Web de Cocina:** Conexión permanente por **WSS (WebSockets)** escuchando el canal `postgres_changes` de la tabla `pedidos`.
• **Backend → Pasarelas de Pago:** HTTPS REST contra el Sandbox oficial de Transbank (código 597055555532) y endpoints mock de BancoEstado.

1.4 ¿Qué patrones de diseño utilizaron y cómo contribuyen a la mantenibilidad, escalabilidad o seguridad?

• **Observer / Pub-Sub:** Utilizado en el motor Realtime de Supabase para notificar a los tableros Kanban cuando un pedido cambia de estado.
• **Store / State Management con Persist:** Implementado en Zustand para mantener el carrito y la sesión aislados y recuperables.
• **Repository / Data Access Layer:** Centralizado en `lib/supabase.ts` para que ningún componente invoque endpoints directamente.
• **Policy-Based Security:** Patrón de seguridad declarativa mediante políticas RLS en PostgreSQL, evaluando permisos fila por fila.

1.5 ¿Qué ocurriría si el número de usuarios aumentara considerablemente? ¿Cómo escala la arquitectura?

"La arquitectura escala en tres capas independientes: los frontends son completamente estáticos y se distribuyen vía CDN global (Cloudflare/Vercel) sin consumir cómputo. La base de datos PostgreSQL escala verticalmente en cómputo/RAM y cuenta con un Connection Pooler transaccional (**PgBouncer**) que gestiona miles de conexiones concurrentes sin saturar el motor. Para picos extremos de demanda, Supabase permite activar réplicas de solo lectura (*Read Replicas*) para distribuir la carga del catálogo."

Bloque 2: Bases de Datos y Gestión de Información

2.1 ¿Qué motor de base de datos utilizaron y cuáles fueron los criterios técnicos para seleccionarlo?

"Utilizamos **PostgreSQL 15** administrado en Supabase. Los criterios técnicos fueron: soporte completo de transacciones ACID, soporte nativo de seguridad a nivel de fila (Row Level Security), extensiones avanzadas (`uuid-ossp`, `pgcrypto`) y capacidad de replicación lógica (CDC) para emisión de eventos en tiempo real."

2.2 ¿Por qué eligieron una base de datos relacional y qué ventajas ofrece para su proyecto?

"Un sistema de compras y recaudación exige **consistencia estricta (cumplimiento ACID)**. Si un alumno compra un producto, el stock, la comanda y la transacción deben consolidarse de forma atómica; no se puede tolerar consistencia eventual. El modelo relacional garantiza integridad referencial entre pedidos, locales y transacciones mediante llaves foráneas."

2.3 ¿Qué diferencias existen entre SQL y NoSQL, y cuándo es recomendable cada una?

• **SQL (Relacional):** Esquema estructurado y rígido, relaciones mediante claves foráneas y cumplimiento ACID. Recomendable para e-commerce, finanzas, ERP y sistemas donde la integridad de datos es crítica.
• **NoSQL (No Relacional):** Esquema flexible (documentos, clave-valor, grafos), escalamiento horizontal masivo y consistencia eventual. Recomendable para analítica de clicks, telemetría IoT, redes sociales y logs masivos.

2.4 ¿Cómo diseñaron su modelo de datos y qué relaciones existen entre las principales entidades?

"El esquema consta de 13 tablas relacionales estructuradas en torno a: `locatarios` (1:N) con `productos` y `categorias_menu`; `usuarios` (1:N) con `pedidos`; `pedidos` (1:N) con `pedido_items` para detallar los platos seleccionados; y `pedidos` (1:1) con `transacciones_pago` para registrar el respaldo financiero. Todas las llaves primarias utilizan UUID v4."

2.5 ¿Qué mecanismos implementaron para garantizar la integridad, consistencia, disponibilidad y seguridad?

• **Integridad:** Llaves foráneas con eliminación restringida o en cascada y tipos ENUM estrictos.
• **Consistencia:** Transacciones atómicas ACID en PostgreSQL.
• **Disponibilidad:** Respaldos automatizados y alta disponibilidad de Supabase en la nube.
• **Seguridad:** Políticas RLS activas en todas las tablas, tokens JWT con expiración y cifrado en tránsito (TLS 1.3) y reposo (AES-256).

Bloque 3: Selección de Tecnologías y Desarrollo

3.1 ¿Qué lenguajes, frameworks y herramientas utilizaron y por qué los seleccionaron?

• **React Native + Expo (TypeScript):** Desarrollo multiplataforma nativo para iOS y Android con una sola base de código.
• **React 19 + Vite 8 (TypeScript + Tailwind CSS 4):** Terminal web ultrarrápido para pantallas de cocina y POS.
• **Supabase + PostgreSQL 15:** Backend serverless con base de datos relacional y WebSockets nativos.
• **Zustand:** Estado global atómico sin el sobrecosto ni boilerplate de Redux.

3.2 ¿Qué otras alternativas tecnológicas evaluaron y por qué las descartaron?

• **Flutter:** Descartado por requerir el lenguaje Dart, fragmentando el stack tecnológico del equipo.
• **Node.js + Express monolítico:** Descartado porque implementar WebSockets, autenticación JWT y réplica de base de datos desde cero habría consumido el 50% del cronograma de desarrollo.
• **MongoDB:** Descartado por no ofrecer integridad referencial nativa ni soporte RLS estricto para órdenes de pago.

3.3 ¿Qué ventajas presenta su stack tecnológico respecto de otras soluciones disponibles?

"La principal ventaja es la **homogeneidad completa en TypeScript**: tanto la aplicación móvil, como la consola web y las interfaces de base de datos comparten contratos de tipos (types compartidos). Además, la recarga en caliente de Vite y Expo aceleró drásticamente la velocidad de desarrollo e integración continua."

3.4 ¿Qué dificultades técnicas enfrentaron durante el desarrollo y cómo las resolvieron?

1. *Recursión en RLS (error 42P17):* Una política de seguridad generó un bucle infinito al consultar la misma tabla en un subquery; se resolvió desanidando la regla y creando funciones con permisos `SECURITY DEFINER`.
2. *Compatibilidad de notificaciones en Expo Go:* Librerías nativas requerían un build de desarrollo compilado; se adaptó un escuchador global WebSocket con `Alert.alert` y `Vibration` nativa sin dependencias externas.
3. *Cálculo distorsionado de SLA:* Pedidos antiguos marcados días después arrojaban miles de minutos de espera; se corrigió sincronizando timestamps UTC con triggers.

3.5 Si tuvieran que desarrollar nuevamente el proyecto, ¿mantendrían las mismas tecnologías o realizarían cambios?

"Mantendríamos el 100% del stack central (React Native, React 19, Supabase y PostgreSQL) por su estabilidad y velocidad. Como única evolución técnica, incorporaríamos **Edge Functions** en Supabase para despachar webhooks de confirmación bancaria en servidores distribuidos geográficamente y soporte para notificaciones push en segundo plano fuera de Expo Go."

Bloque 4: Metodologías de Desarrollo y Gestión del Proyecto

4.1 ¿Qué metodología de desarrollo utilizaron y por qué la consideraron adecuada?

"Utilizamos **Scrumban**. Nos proporcionó la estructura de iteraciones fijas de 2 semanas de Scrum para cumplir los hitos académicos de Duoc UC, combinada con la flexibilidad de Kanban para visualizar el flujo continuo mediante límites WIP (Work In Progress), adaptándonos ágilmente ante bloqueos técnicos en la integración de pasarelas y base de datos."

4.2 ¿Cuáles son las principales diferencias entre una metodología ágil y una tradicional (predictiva)?

• **Tradicional (Cascada):** Planificación rígida al inicio; fases secuenciales cerradas; no hay software funcional hasta el final; los cambios de alcance son costosos y traumáticos.
• **Ágil (Scrum/Scrumban):** Planificación adaptativa; entregas funcionales frecuentes e incrementales; el alcance se ajusta continuamente según la retroalimentación de los usuarios y pruebas técnicas.

4.3 ¿Qué ventajas y desventajas tendría utilizar Scrum en lugar de un modelo en cascada para este proyecto?

• **Ventajas:** Permitió tener un flujo móvil funcional en la semana 10 (Hito 2), detectar defectos de RLS tempranamente y ajustar la arquitectura a React 19 + Vite sin rehacer la documentación previa.
• **Desventajas:** Requiere una disciplina rigurosa de reuniones diarias (Dailies) y puede generar incertidumbre en la estimación de tiempos si los requerimientos cambian con frecuencia.

4.4 ¿Cómo organizaron las actividades, asignaron responsabilidades y realizaron el seguimiento?

"Utilizamos **GitHub Projects** vinculado al repositorio de código. El trabajo se dividió bajo el principio de 'un dueño por módulo' para evitar colisiones. Para el control de versiones aplicamos **Git Flow** con la rama `main` protegida: ninguna línea de código entraba sin una rama de feature y un Pull Request con revisión cruzada obligatoria."

4.5 ¿Cómo gestionaron los cambios en los requerimientos y qué impacto tuvieron en el cronograma?

"Los cambios se gestionaron mediante la repriorización del **Product Backlog**. Cuando se identificó la necesidad del ticket térmico de 80 mm para locales sin pantalla en cocina, se absorbió como una historia de usuario secundaria en el Sprint 3 sin alterar el camino crítico del cronograma ni la fecha de entrega final de 18 semanas."

Bloque 5: Pruebas de Software y Aseguramiento de la Calidad (QA)

5.1 ¿Qué tipos de pruebas realizaron y por qué seleccionaron esas pruebas?

• **Pruebas Funcionales y de Sistema:** Validar que cada requerimiento (crear pedido, cambiar estado) opere según lo especificado.
• **Pruebas de Integración:** Verificar el flujo E2E entre App Móvil, Supabase y Consola Web.
• **Pruebas de Seguridad:** Comprobar que un locatario no pueda leer datos ajenos mediante inyección o tokens alterados (validación de RLS).
• **Pruebas de Regresión:** Reejecutar pruebas tras corregir defectos para garantizar que el parche no introdujera nuevas fallas.

5.2 ¿Cuántos casos de prueba diseñaron, cuántos ejecutaron y qué porcentaje resultó exitoso?

"Se diseñaron **20 casos de prueba formales (CP-01 a CP-20)**. Se ejecutaron los 20 casos (100% de ejecución). Al término de los 4 ciclos, los 20 casos resultaron exitosos, alcanzando una **tasa de éxito del 100%** sin defectos bloqueantes residuales."

5.3 ¿Cuántos ciclos de pruebas realizaron y qué actividades desarrollaron en cada ciclo?

- **Ciclo 1 (Semana 14):** Pruebas de núcleo, base de datos, autenticación y seguridad RLS.
- **Ciclo 2 (Semana 15):** Flujo E2E de pedidos, notas de cocina y estados del Kanban.
- **Ciclo 3 (Semana 15):** Modo oscuro, diseño responsivo y accesibilidad UI/UX.
- **Ciclo 4 (Semana 16):** Limpieza de mocks, pasarela de pago Sandbox y pruebas de regresión final.

5.4 ¿Cuántos defectos detectaron durante las pruebas y cómo los clasificaron?

"Se detectaron un total de **11 defectos**. Se clasificaron según su severidad: 3 Críticos / Bloqueantes (destacando la recursión en RLS 42P17 y fallos de token), 5 Mayores (errores en el cálculo de subtotal y notas de cocina) y 3 Menores (ajustes de espaciado y contraste en modo oscuro). El **100% de los defectos fue corregido**, quedando 0 defectos pendientes."

5.5 ¿Qué evidencias permiten demostrar que el sistema cumple con los requerimientos?

"La evidencia principal es la **Matriz de Trazabilidad de Requisitos (RTM)** contenida en el `01_PLAN_DE_PRUEBAS.docx`, que mapea el 100% de los Requisitos Funcionales (RF-01 al RF-14) contra los casos de prueba ejecutados. Se complementa con las actas de ejecución con capturas en el `02_CASOS_DE_PRUEBA.docx` y la demostración en video del software completo operando en vivo."

Bloque 6: Preguntas Complementarias de QA y Certificación

6.1 ¿Cuál fue el criterio utilizado para determinar la cantidad de casos de prueba?

"El criterio fue la **cobertura total de requisitos comprometidos**. Para cada uno de los 14 Requisitos Funcionales se diseñó al menos un caso de prueba para el flujo principal (*Happy Path*) y casos adicionales para flujos alternativos o de excepción (ej. autenticación con credenciales inválidas, stock agotado o cancelación de pedido), resultando en los 20 casos de prueba."

6.2 ¿Por qué realizaron 4 ciclos de pruebas y no más o menos?

"Porque 4 ciclos correspondieron a la madurez progresiva del producto siguiendo el ciclo de Deming (Plan-Do-Check-Act). Menos ciclos habría significado certificar sobre código inestable; más ciclos habría resultado redundante, ya que en el Ciclo 4 se ejecutaron los 20 casos con 0 defectos detectados, alcanzando la meseta de estabilización."

6.3 ¿Qué diferencias encontraron entre los resultados del primer y último ciclo de pruebas?

"En el primer ciclo se identificó una tasa de fallo del 30%, con 6 defectos graves concentrados en bloqueos de base de datos por RLS y desbordes de memoria en WebSockets. En el cuarto ciclo, tras aplicar los parches, la tasa de éxito fue del 100%, con latencias de comanda inferiores a 500 ms y respuesta fluida en todos los dispositivos."

6.4 ¿Qué pruebas ejecutaron nuevamente después de corregir los defectos identificados?

"Ejecutamos **pruebas de regresión focalizadas** en los módulos afectados y sus dependencias directas. Por ejemplo, al corregir el error de recursión en RLS 42P17, se reejecutaron no solo las pruebas de creación de pedidos, sino también las de consulta de catálogo, login institucional y cuadratura de caja para descartar efectos colaterales."

6.5 ¿Qué criterios utilizaron para determinar que el sistema estaba listo para su entrega (Definition of Done)?

1. 100% de los casos de prueba ejecutados y con resultado Aprobado.
2. Cero defectos de severidad Crítica o Mayor abiertos en el registro.
3. Matriz de Trazabilidad RTM completa y validada.
4. Checklist de Término de Proyecto firmado con evidencias comprobadas de código fuente y documentación homologada.